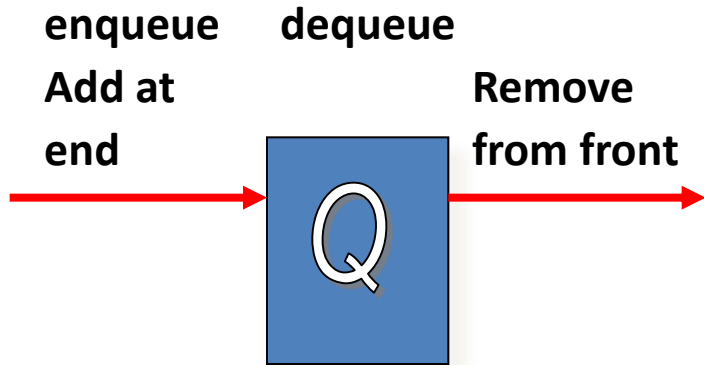


Heaps & Heap Sort

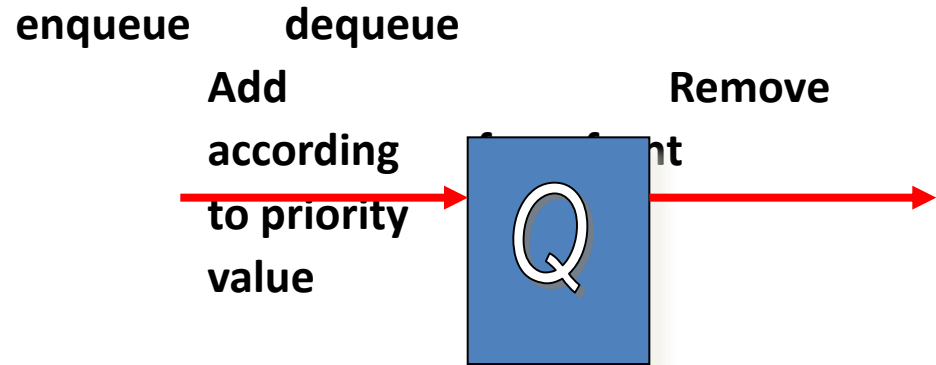
Priority Queues

A queue that is ordered according to some priority value

Standard Queue



Priority Queue



Applications of Priority Queues

Line-up of Incoming Planes at Airport

Possible Criteria for Priority?

Operating Systems Priority Queues?

Several criteria could be mapped to a priority status

Max-Priority Queue Operations

Insert(S, x) – Inserts element x into set S , according to its priority

Maximum(S) – Returns, but does not remove, the element of S with the largest key

Extract-Max(S) – Removes and returns the element of S with the largest key

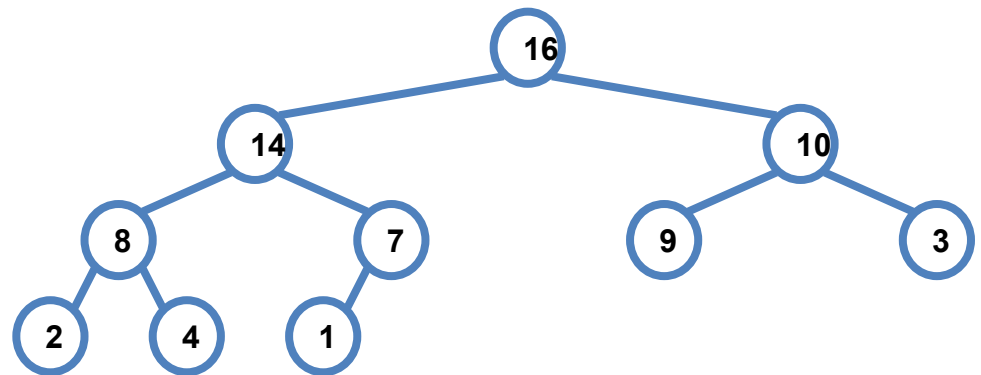
Increase-Key(S, x, k) – Increases the value of element x 's key to the new value k

Binary Heaps

- The (binary) heap data structure is an array object that can be viewed as a complete binary tree
 - Each node of the tree corresponds to an element of the array that stores the value in the node.
 - The tree is completely filled on all levels except possibly the lowest, where it is filled from the left up to a point.

$A =$

16	14	10	8	7	9	3	2	4	1
----	----	----	---	---	---	---	---	---	---



Binary Heaps

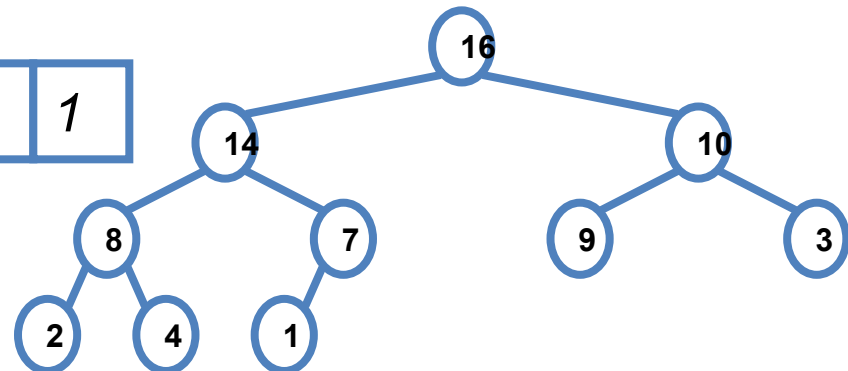
□ To represent a complete binary tree as an array:

- The root node is $A[1]$
- Node i is $A[i]$
- The parent of node i is $A[i/2]$
- The left child of node i is $A[2i]$
- The right child of node i is $A[2i + 1]$

```
Parent(i)  
    return floor(i/2)
```

```
Right(i)  
    return 2i+1
```

```
Left(i)  
    return 2i
```



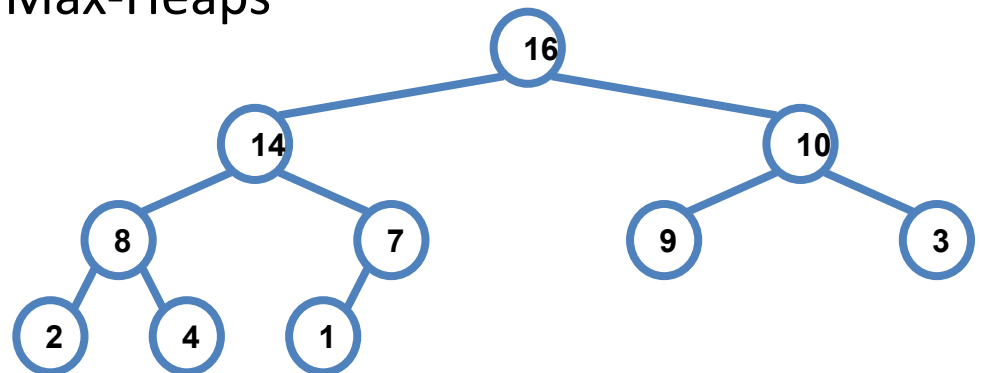
Types of Binary Heaps

- **Min-Heaps:**

- The element in the root is less than or equal to all elements in both of its sub-trees
- Both of its sub-trees are Min-Heaps

- **Max-Heaps:**

- The element in the root is greater than or equal to all elements in both its sub-trees
- Both of its sub-trees are Max-Heaps



The Heap Property

- Max-Heaps satisfy the *heap property*:

$$A[\text{Parent}(i)] \geq A[i] \quad \text{for all nodes } i > 1$$

- In other words, the value of a node is at most the value of its parent
- The largest element in a max-heap is stored at the root

- Min-Heaps satisfy the *heap property*:

$$A[\text{Parent}(i)] \leq A[i] \quad \text{for all nodes } i > 1$$

- In other words, the value of a node is at least the value of its parent
- The smallest element in a min-heap is stored at the root

Max-Heap Operations: Max-Heapify()

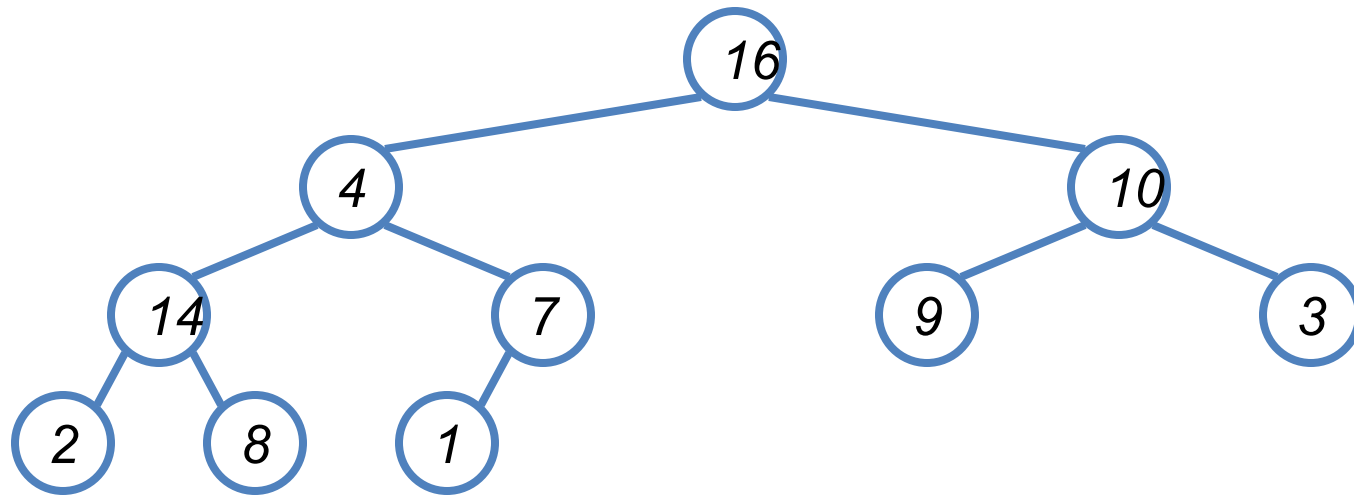
- **Max-Heapify()** : maintain the max-heap property
 - Given: a node i in the heap with children l and r
 - Given: two subtrees rooted at l and r , assumed to be heaps
 - Problem: The subtree rooted at i may violate the heap property (*How?*)
 - Action: let the value of the parent node “float down” so subtree at i satisfies the heap property
 - *What do you suppose will be the basic operation between i , l , and r ?*

Max-Heap Operations: Max-Heapify()

Assumed that the binary trees rooted at $\text{LEFT}(i)$ and $\text{RIGHT}(i)$ are already max-heaps.

```
MAX-HEAPIFY( $A, i$ )
1   $l \leftarrow \text{LEFT}(i)$ 
2   $r \leftarrow \text{RIGHT}(i)$ 
3  if  $l \leq \text{heap-size}[A]$  and  $A[l] > A[i]$ 
4      then  $\text{largest} \leftarrow l$ 
5      else  $\text{largest} \leftarrow i$ 
6  if  $r \leq \text{heap-size}[A]$  and  $A[r] > A[\text{largest}]$ 
7      then  $\text{largest} \leftarrow r$ 
8  if  $\text{largest} \neq i$ 
9      then exchange  $A[i] \leftrightarrow A[\text{largest}]$ 
10     MAX-HEAPIFY( $A, \text{largest}$ )
```

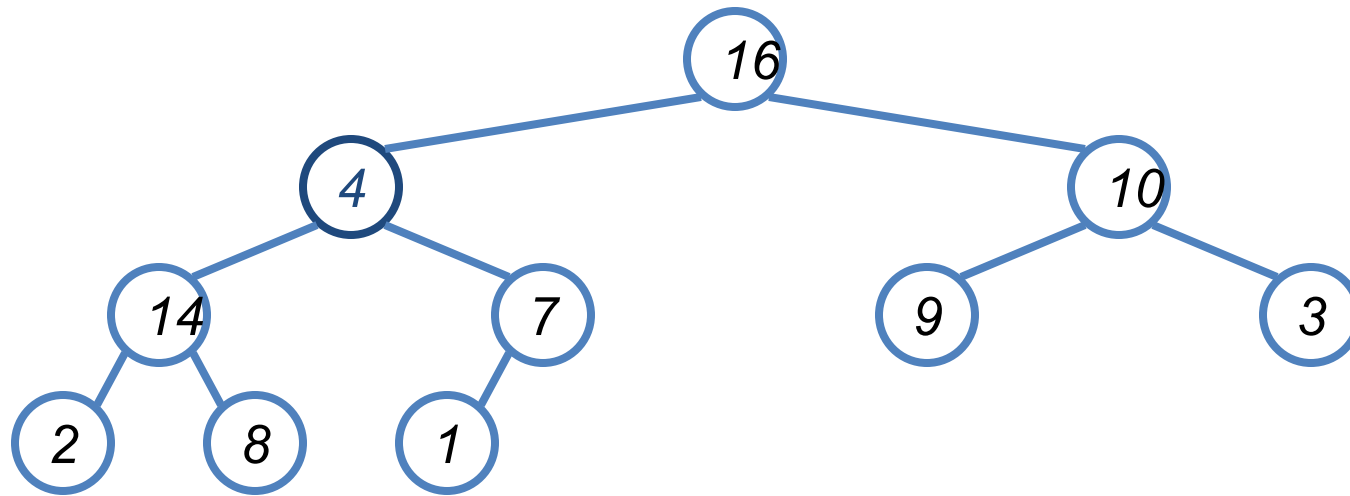
Heapify() Example



$A =$

16	4	10	14	7	9	3	2	8	1
----	---	----	----	---	---	---	---	---	---

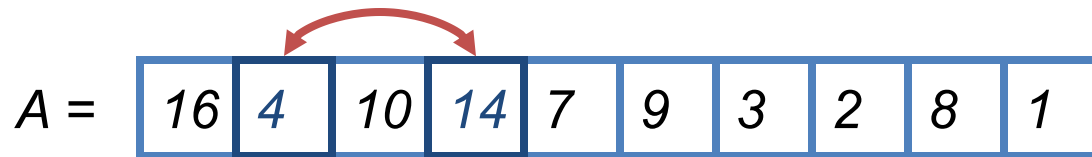
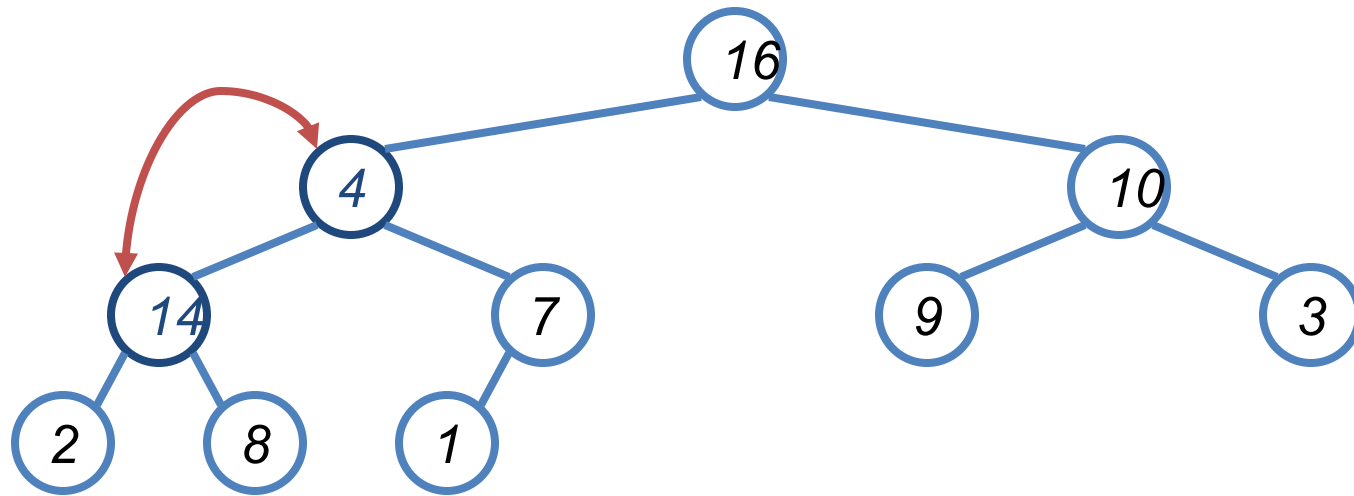
Heapify() Example



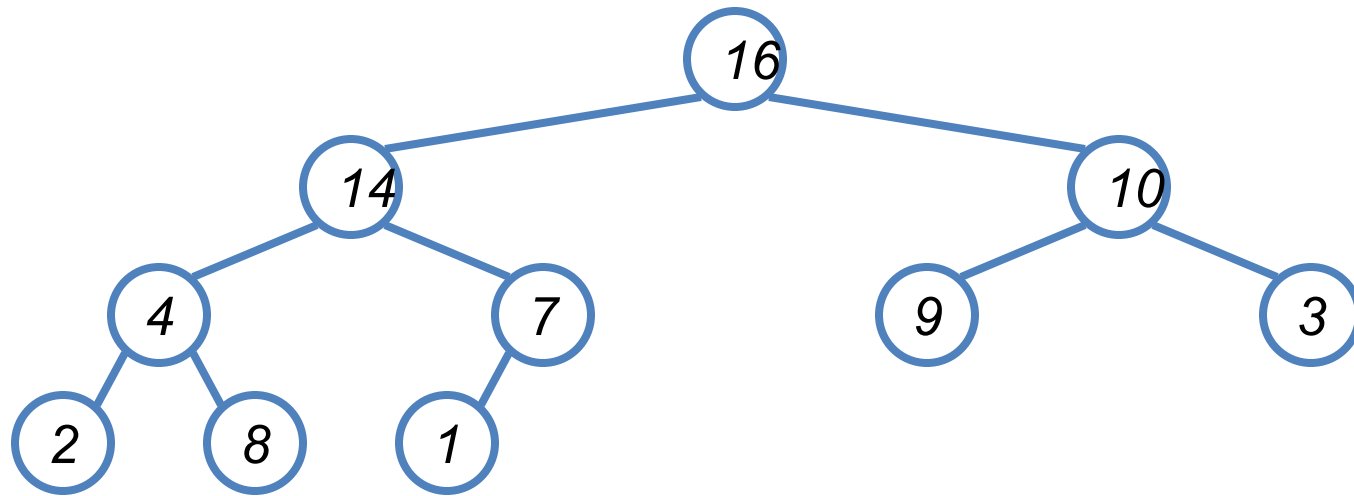
$A =$

16	4	10	14	7	9	3	2	8	1
----	---	----	----	---	---	---	---	---	---

Heapify() Example



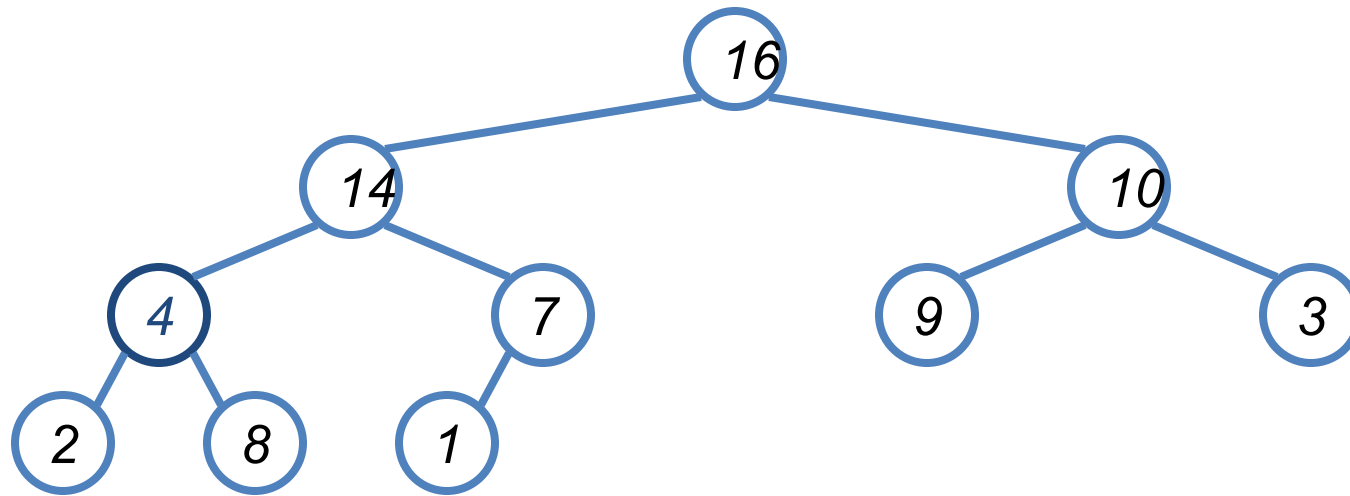
Heapify() Example



$A =$

16	14	10	4	7	9	3	2	8	1
----	----	----	---	---	---	---	---	---	---

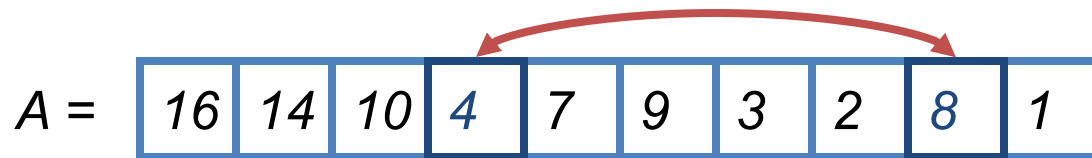
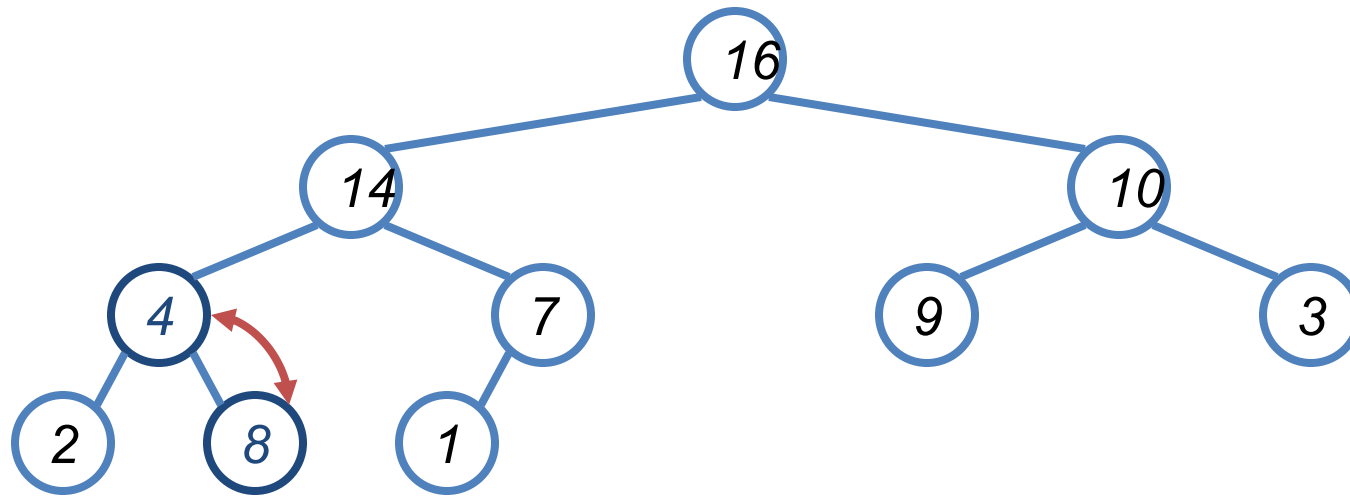
Heapify() Example



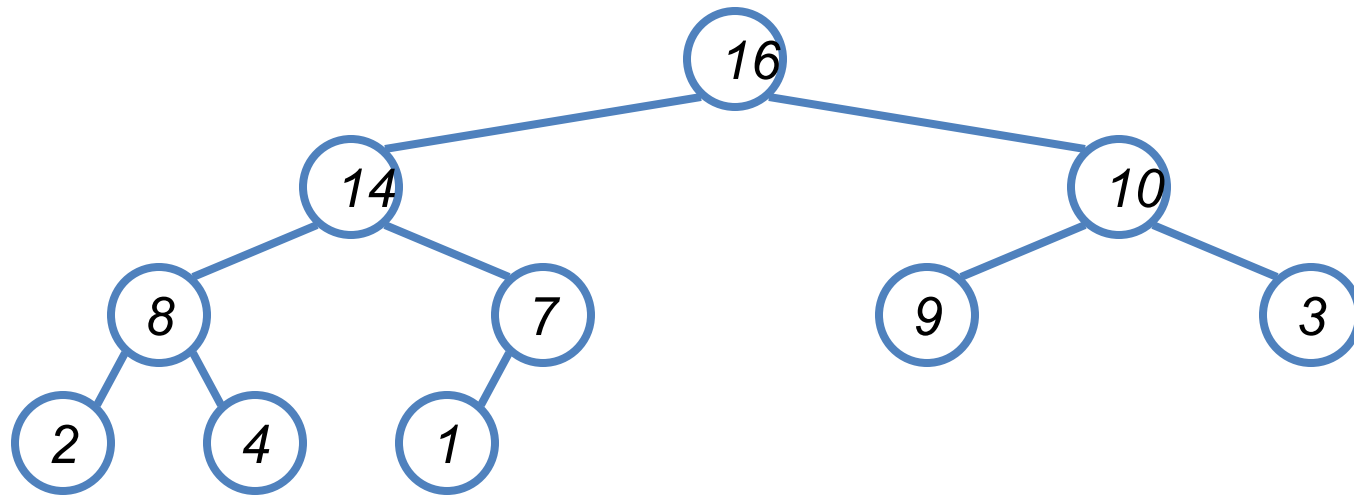
$A =$

16	14	10	4	7	9	3	2	8	1
----	----	----	---	---	---	---	---	---	---

Heapify() Example



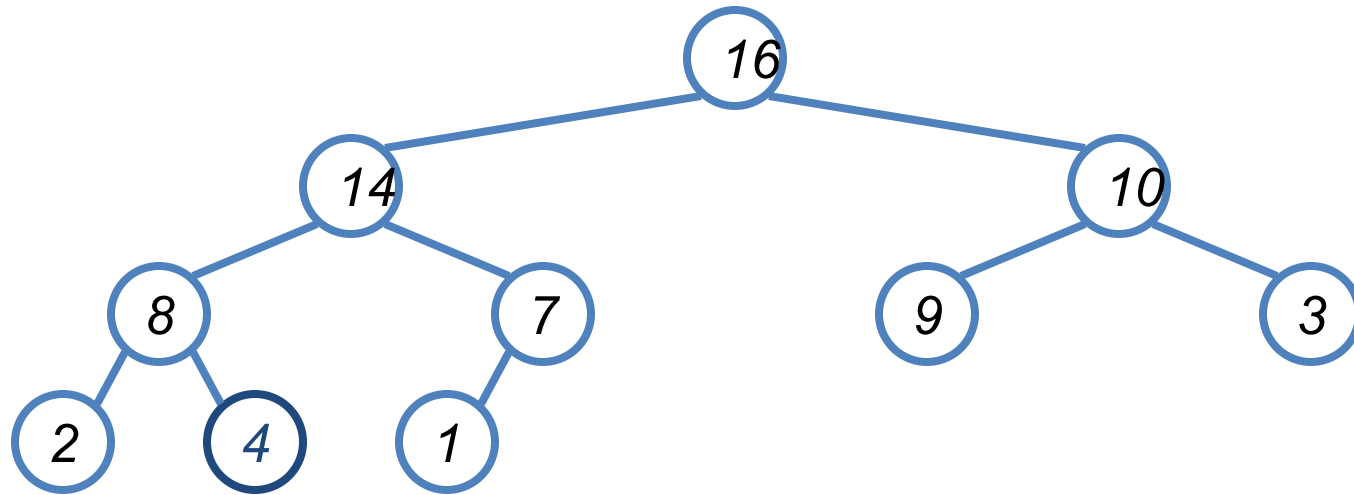
Heapify() Example



$A =$

16	14	10	8	7	9	3	2	4	1
----	----	----	---	---	---	---	---	---	---

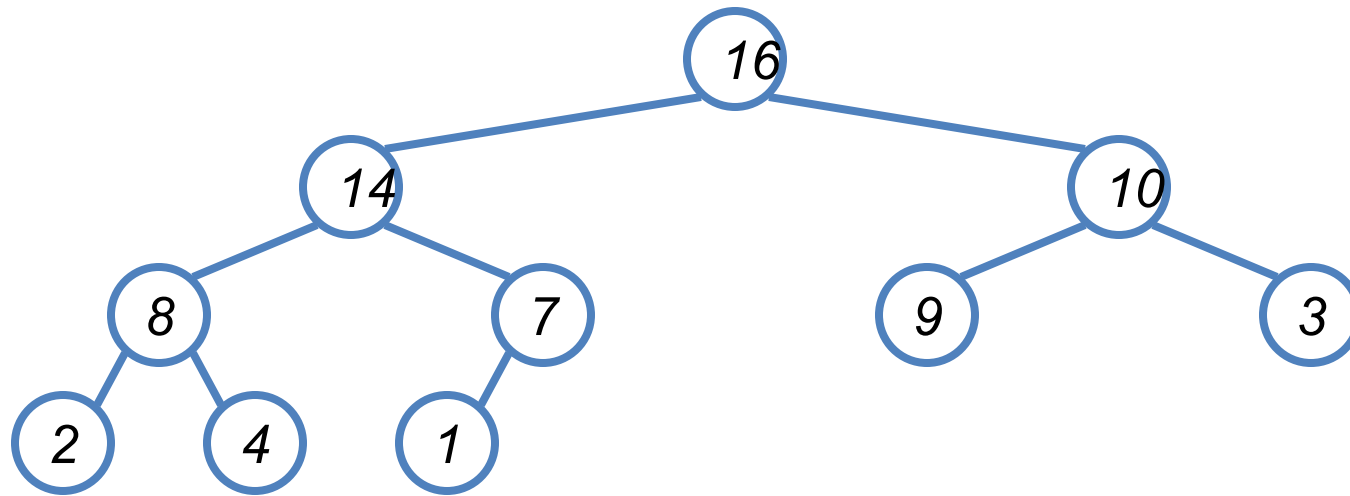
Heapify() Example



$A =$

16	14	10	8	7	9	3	2	4	1
----	----	----	---	---	---	---	---	---	---

Heapify() Example



$A =$

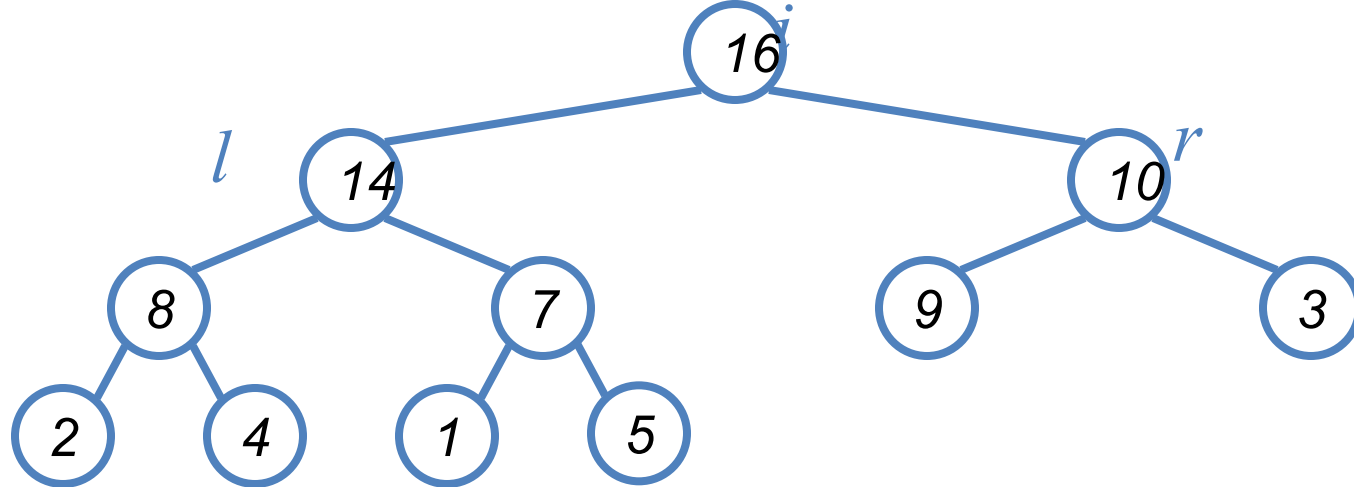
16	14	10	8	7	9	3	2	4	1
----	----	----	---	---	---	---	---	---	---

Analyzing Heapify(): Informal

- *Aside from the recursive call, what is the running time of **Heapify()**?*
- *How many times can **Heapify()** recursively call itself?*
- *What is the worst-case running time of **Heapify()** on a heap of size n ?*

Analyzing Heapify(): Formal

- Fixing up relationships among the elements $A[i]$, $A[l]$, and $A[r]$ takes $\Theta(1)$ time
- *If the heap at i has n elements, at most how many elements can the subtrees at l or r have?*



- Answer: $2n/3$ (worst case: bottom row half full)
- So time taken by **Heapify()** is given by
$$T(n) \leq T(2n/3) + \Theta(1)$$

Analyzing Heapify(): Formal

- So we have

$$T(n) \leq T(2n/3) + \Theta(1)$$

- Solving the recurrence, we have

$$T(n) = O(\lg n)$$

- Thus, **Heapify** () takes $O(h)$ time for a node at height h .

Heap Operations: BuildHeap()

- We can build a heap in a bottom-up manner by running **Heapify()** on successive subarrays
 - Fact: for array of length n , all elements in the range $A[\lfloor n/2 \rfloor + 1 \dots n]$ are heaps (*Why?*)
 - So:
 - Walk backwards through the array from $n/2$ to 1, calling **Heapify()** on each node.
 - Order of processing guarantees that the children of node i are heaps when i is processed

Heap Operations: BuildHeap()

Converts an unorganized array A into a max-heap.

BUILD-MAX-HEAP(A)

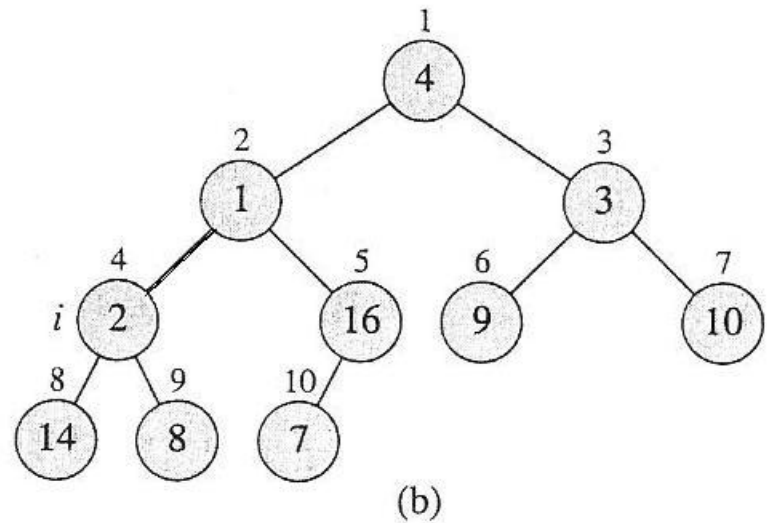
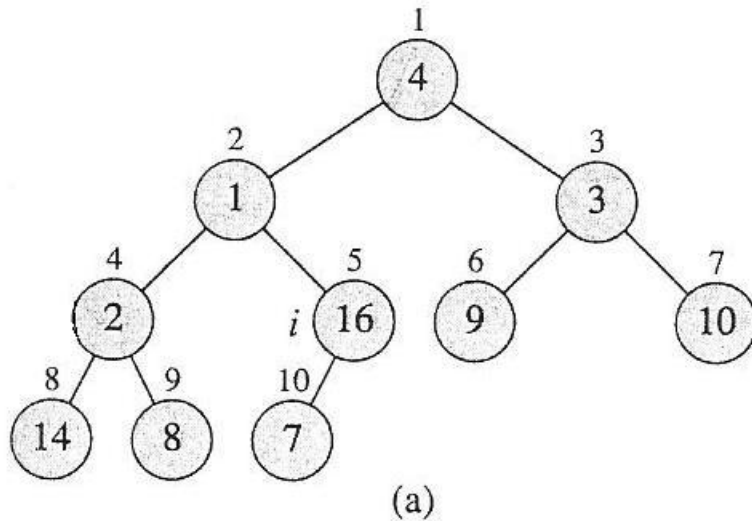
```
1  heap-size[ $A$ ]  $\leftarrow$  length[ $A$ ]  
2  for  $i \leftarrow \lfloor \text{length}[A]/2 \rfloor$  downto 1  
3      do MAX-HEAPIFY( $A, i$ )
```

BuildHeap() Example

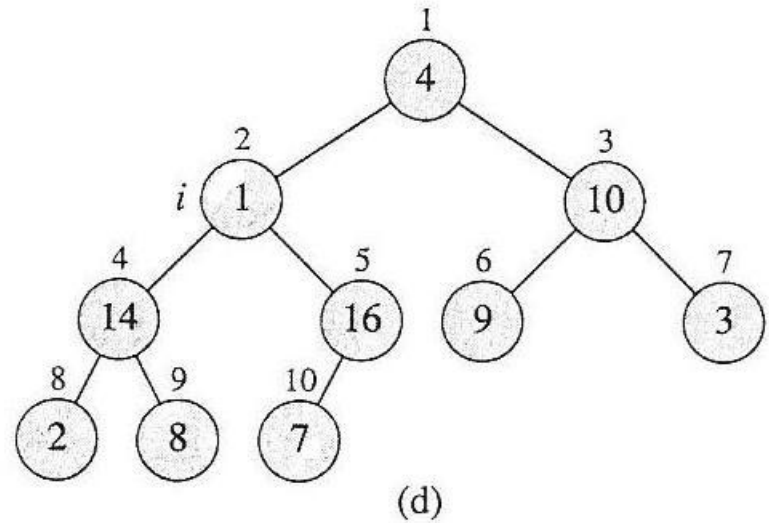
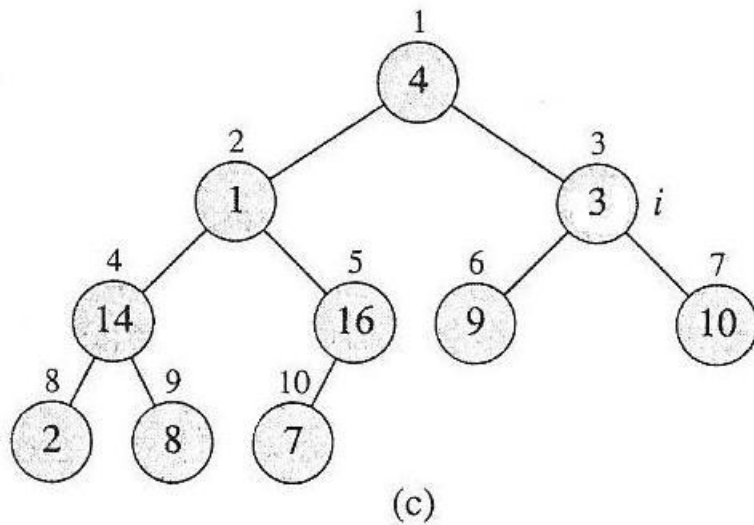
- Work through example

A

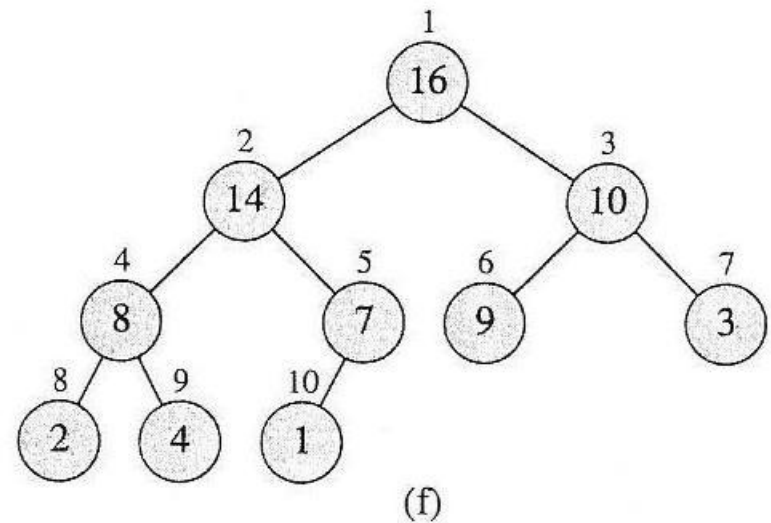
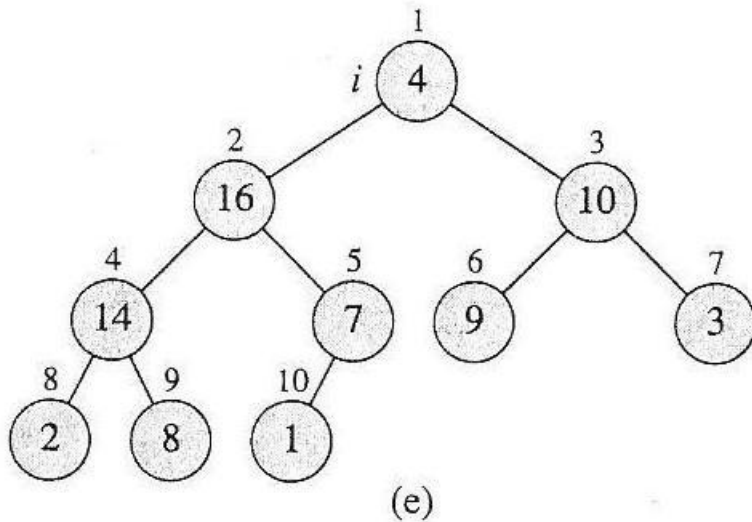
4	1	3	2	16	9	10	14	8	7
---	---	---	---	----	---	----	----	---	---



BuildHeap() Example



BuildHeap() Example



Heapsort

- Given **BuildHeap()**, a sorting algorithm can easily be constructed:
 - Maximum element is at $A[1]$
 - Discard by swapping with element at $A[n]$
 - Decrement $\text{heap_size}[A]$
 - $A[n]$ now contains correct value
 - Restore heap property at $A[1]$ by calling **Heapify()**
 - Repeat, always swapping $A[1]$ for $A[\text{heap_size}(A)]$

Heapsort

Heapsort(A)

```
{  
    BuildHeap(A);  
    for (i = length(A) downto 2)  
    {  
        Swap(A[1], A[i]);  
        heap_size(A) = heap_size(A) - 1;  
        Heapify(A, 1);  
    }  
}
```

Priority Queues

- Heapsort is a nice algorithm, but in practice Quicksort (coming up) usually wins
- The heap data structure is incredibly useful for implementing *priority queues*
 - A data structure for maintaining a set S of elements, each with an associated value or *key*
 - Supports the operations **Insert()**, **Maximum()**, and **ExtractMax()**

Priority Queue Operations

$\text{Insert}(S, x)$ – Inserts element x into set S , according to its priority

$\text{Maximum}(S)$ – Returns, but does not remove, the element of S with the largest key

$\text{Extract-Max}(S)$ – Removes and returns the element of S with the largest key

$\text{Increase-Key}(S, x, k)$ – Increases the value of element x 's key to the new value k

□ *How could we implement these operations using a heap?*

Priority Queue Operations

HEAP-MAXIMUM(A)

1 **return** $A[1]$

HEAP-EXTRACT-MAX(A)

```
1  if  $heap-size[A] < 1$ 
2      then error “heap underflow”
3   $max \leftarrow A[1]$ 
4   $A[1] \leftarrow A[heap-size[A]]$ 
5   $heap-size[A] \leftarrow heap-size[A] - 1$ 
6  MAX-HEAPIFY( $A, 1$ )
7  return  $max$ 
```

Priority Queue Operations

HEAP-INCREASE-KEY(A, i, key)

```
1  if  $key < A[i]$ 
2      then error “new key is smaller than current key”
3   $A[i] \leftarrow key$ 
4  while  $i > 1$  and  $A[PARENT(i)] < A[i]$ 
5      do exchange  $A[i] \leftrightarrow A[PARENT(i)]$ 
6       $i \leftarrow PARENT(i)$ 
```

Priority Queue Operations

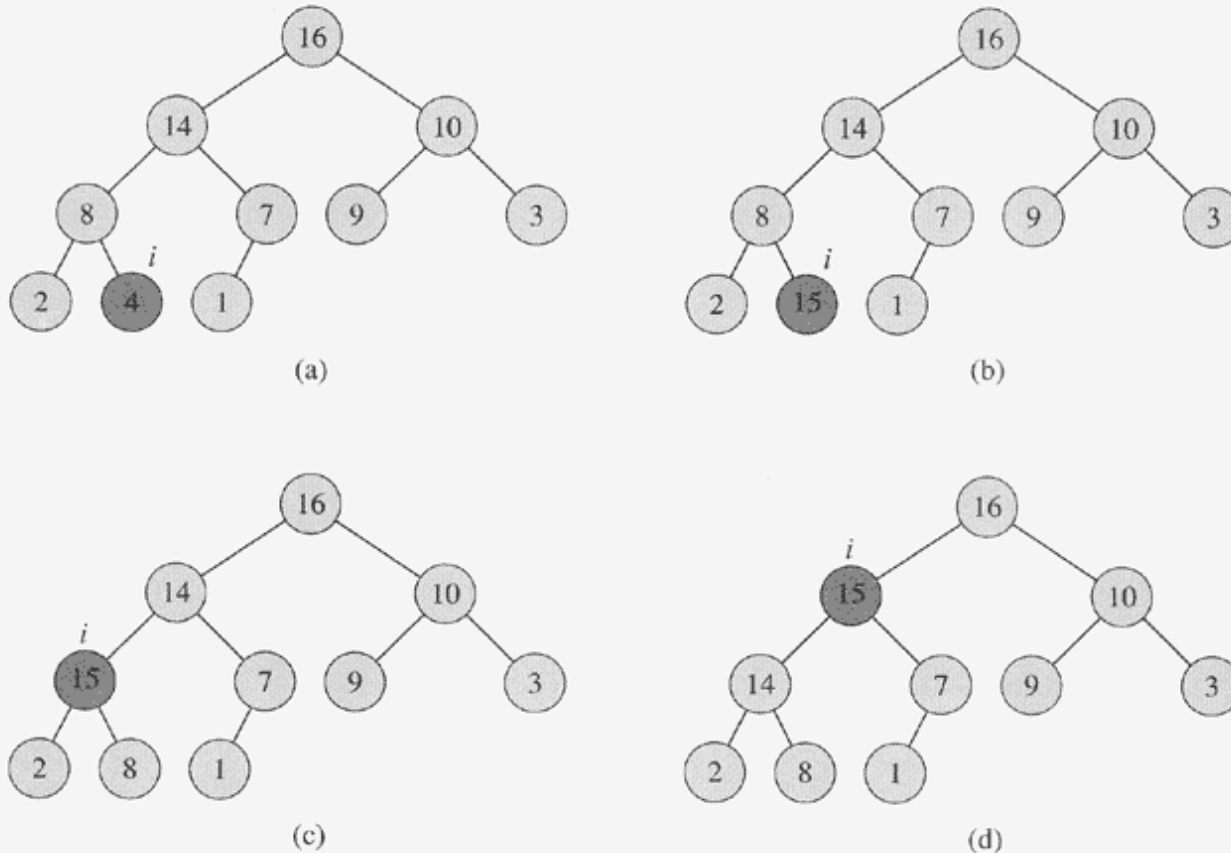


Figure 6.5 The operation of HEAP-INCREASE-KEY. (a) The max-heap of Figure 6.4(a) with a node whose index is i heavily shaded. (b) This node has its key increased to 15. (c) After one iteration of the **while** loop of lines 4–6, the node and its parent have exchanged keys, and the index i moves up to the parent. (d) The max-heap after one more iteration of the **while** loop. At this point, $A[\text{PARENT}(i)] \geq A[i]$. The max-heap property now holds and the procedure terminates.

Priority Queue Operations

MAX-HEAP-INSERT(A, key)

1 $heap-size[A] \leftarrow heap-size[A] + 1$

2 $A[heap-size[A]] \leftarrow -\infty$

3 HEAP-INCREASE-KEY($A, heap-size[A], key$)